

Phase 8 – Cache All Products (Redis)

Objective

Improve application performance by caching the complete list of products in Redis, reducing repeated database queries for `GET /products`.

Why Cache the Product List?

Initially, only individual products were cached.

Example:

```
GET /products/1
```

Redis Key:

```
product:1
```

However, when fetching all products:

```
GET /products
```

the application always queried MySQL:

```
SELECT * FROM product;
```

This meant Redis wasn't helping with the product list endpoint.

To solve this, we introduced a dedicated cache entry:

```
products:all
```

which stores the complete list of products.

Redis Keys

Single Product

```
product:1  
product:2  
product:3  
...
```

TTL:

```
5 Minutes
```

Product List

```
products:all
```

TTL:

```
2 Minutes
```

Redis Constants

Created a constants class to centralize cache keys and TTL values.

```
public final class RedisConstants {  
  
    private RedisConstants() {}  
  
    public static final String PRODUCT_KEY_PREFIX = "product:";  
    public static final Duration PRODUCT_CACHE_TTL = Duration.ofMinutes(5);  
  
    public static final String PRODUCT_LIST_KEY = "products:all";  
    public static final Duration PRODUCT_LIST_TTL = Duration.ofMinutes(2);  
}
```

Benefits:

- No hardcoded strings
 - Easy maintenance
 - Centralized cache configuration
-

ProductRedisService

Added support for caching the complete product list.

Methods:

```
List<Product> getAllProducts();  
  
void saveAllProducts(List<Product> products);  
  
void deleteAllProductsCache();
```

Responsibilities:

- Read list from Redis
 - Save list into Redis
 - Delete list cache
-

ProductService Flow

GET /products

Step 1

Check Redis.

```
products:all
```

If present:

```
Return cached list.
```

If absent:

Fetch from Database.

Then

Save into Redis.

Finally

Return response.

Pseudo Flow:

Redis

↓

Cache Hit?

↓

Yes → Return Cached List

↓

No

↓

Database

↓

Save List to Redis

↓

Return List

Cache Invalidation

Whenever data changes, the list cache becomes stale.

Therefore, after:

- Create Product
- Update Product
- Delete Product

we invalidate:

```
products:all
```

Example:

```
redisService.deleteAllProductsCache();
```

This ensures that the next request reloads the latest data from MySQL.

Runtime Flow

First Request

```
GET /products
```

Logs:

```
Cache MISS
```

```
↓
```

```
Database Query
```

```
↓
```

```
Store products:all
```

```
↓
```

```
Return List
```

Hibernate executes:

```
SELECT * FROM product;
```

Redis now contains:

```
products:all
```

Second Request

```
GET /products
```

Logs:

```
Cache HIT
```

```
↓
```

```
Return Product List from Redis
```

No database query is executed.

Redis Structure

```
Redis
```

```
├─ product:1
```

```
├─ product:2
```

```
├─ product:3
```

```
└─ products:all
```

Each single product has its own cache entry.

The complete product list is stored separately.

Logging

Implemented meaningful logs for learning and debugging.

Examples:

```
Received request to fetch all products
```

```
Cache MISS : products:all
```

```
Fetching Product List from Database
```

```
Stored Product List into Redis
```

```
Returning Product List from Redis
```

These logs clearly show whether data is being served from Redis or MySQL.

Benefits Achieved

- ✓ Reduced database load
 - ✓ Faster response time
 - ✓ Automatic cache refresh after data changes
 - ✓ Separate caching strategy for single products and product lists
 - ✓ Configurable TTL
 - ✓ Easy-to-maintain Redis key management
-

Redis Concepts Learned So Far

- Redis as an in-memory cache
 - RedisTemplate
 - Cache keys
 - TTL (Time-To-Live)
 - Cache Hit
 - Cache Miss
 - Manual cache eviction
 - Cache invalidation
 - Cache-Aside Pattern
 - Single object caching
 - Collection (List) caching
 - Logging for cache operations
 - Redis key naming conventions
-

Project Progress

Completed

- Spring Boot + MySQL setup
 - Docker Compose (MySQL + Redis + Product Service)
 - CRUD APIs
 - Redis configuration
 - RedisTemplate configuration
 - Manual Redis caching
 - Single Product Cache
 - Product List Cache
 - TTL configuration
 - Cache invalidation
 - Manual cache eviction
 - Exception handling
 - Detailed application logging
-

Next Learning Phase

Phase 9 – Advanced Redis Features

We'll continue exploring Redis with more production-oriented topics such as:

- Redis Hashes
- Redis Sets

- Redis Sorted Sets
- Redis Pub/Sub
- Distributed Locks
- Session Storage
- Spring Cache Abstraction (`@Cacheable` , `@CacheEvict` , `@CachePut`)
- Cache Preloading (Warm-up)
- Monitoring Redis
- Performance optimization and best practices